Programação Orientada a Objetos

Aula 05: Arrays e ArrayLists

Prof. Fabricio Santana

2025.1

fabricio.santana@idp.edu.br

www.linkedin.com/in/fabriciofsantana

O que estudamos até aqui?

Arrays

- **Estrutura de dados**: coleção de dados relacionados

- **Array:** estruturas de dados que consistem em itens de dados relacionados do **mesmo tipo**.
  - Composto por elementos ou componentes
  - É um grupo de variáveis contendo valores do mesmo tipo
    - *Primitive types* ou *Reference Types*
  - Uma vez criado, permanecem do mesmo tamanho
  - Para aumentar o tamanho, deve-se criar um novo array
  - Índice: posição de um elemento no array

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Representação Lógica

- *Array-access expression.*
  - *nome_do_array[indice_do_elemento]*

- O primeiro elemento de um array está no índice 0
  - O índice deve ser um inteiro não negativo

- O método ***length*** retorna o tamanho do array

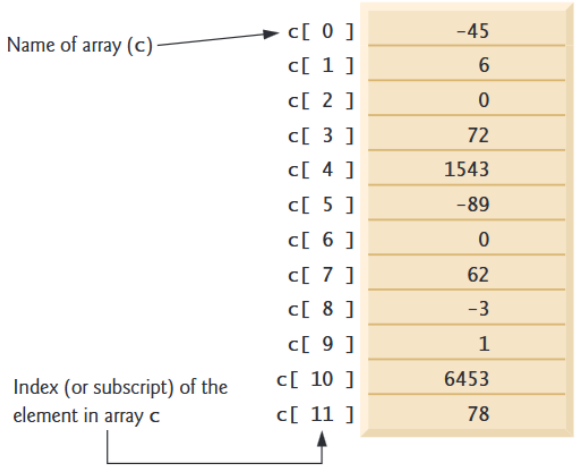

Fig. 7.1 | A 12-element array.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Como declarar um array?

- Array é um objeto: deve ser criado com a palavra-chave **new**
  - **int[] c = new int[12];** **OU** int c[] = new int=[12]; **OU**

  - int[] c; // declaração da variável
  - c = new int[12]; // instanciação do array e atribuição à variável

- Quando um array é criado, cada um de seus elementos recebe um valor padrão:
  - **zero** para os elementos do tipo primitivo numérico,
  - **false** para elementos booleanos e
  - **null** para *reference types*.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Como declarar um array?

- Um programa pode declarar arrays de qualquer tipo
  - os tipos dos elementos do array, seguem o tipo declarao para o array
    - String[] meuArrayDeString = new String[100];
      - Todos os 100 elementos do array serão do tipo String
    - meyArrayDeString[10]: valor do elemento no índice 10
      - **Nunca se esqueça que o array começa do índice zero**
      - Tentar acessar um elemento fora dos limites do array dispara ArrayIndexOutOfBoundsException
- Inicialização de um array: int[] n = {10, 20, 30, 40, 50};

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

É meme! Em java, Array começa do índice 0

https://www.reddit.com/r/ProgrammerHumor/

Tratamento de exceção

- ***Exception***: indica um problema (comportamento indesejado) durante a execução de um programa

- ***Exception handling*** (tratamento de exceção): ajuda a criar programa tolerantes a falhas

- Quando ocorre um problema durante a execução de um programa, a JVM ou um método dispara (**throws**) uma exceção

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Tratamento de exceção

- ***Exception***: indica um problema (comportamento indesejado) durante a execução de um programa

- ***Exception handling*** (tratamento de exceção): ajuda a criar programa tolerantes a falhas

- Quando ocorre um problema durante a execução de um programa, a JVM ou um método dispara (**throws**) uma exceção

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Tratamento de exceção

- **try block**: contém código que pode disparar uma exceção
- ***catch block***: contém código que trata a exceção se ela ocorrer

```
try{
    System.out.println(students[outOfBoundIndex]);
} catch (ArrayIndexOutOfBoundsException e){
    System.out.printf(format:"Student %s does not exist%n", outOfBoundIndex);
    System.out.println(e);
}

System.out.println(x:"Program continues execution ...");
```

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Enhanced for statement

- Permite iterar por um array sem usar um contador
- Evita tentativa de acesso a elemento inexistente (*out of bounds*)

```
for (parameter : arrayName) {
    statement
}
```

- Só pode ser usada para obter elementos de array, ou seja, não pode ser usada para modificar elementos

- Não é possível acessar ou manipular o índice do elemento

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Passando arrays como parâmetros

- É possível passar/receber um array como parâmetro de um método
  - int[] array = {1, 2};
  - modifyArray(array);
  - public static void modifyArray(int[] array)

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Passando arrays como parâmetros

- Tipo de passagem de parâmetro
  - Array
    - o método chamado recebe uma referência para o array original
  - Elemento individual de um array de *reference type*
    - o método chamado recebe uma cópia da referência para o array original
  - Elemento individual de um array de tipo primitivo
    - o método chamado recebe uma cópia do valor

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Passando arrays como parâmetros

- Tipo de passagem de parâmetro: **por valor x por referência**
  - Em java todos parâmetros são passados por valor

  - **por valor:**
    - Uma cópia do valor da variável é passada para o método chamado
    - O método chamado trabalha exclusivamente com a cópia
    - Alterações na cópia do método chamado não afetam o valor da variável original no chamador.

  - **por referência:**
    - o método chamado pode acessar o valor do variável no chamador diretamente e modificar esses dadosmelhora o desempenho ao eliminar a necessidade de copiar possivelmente grandes quantidades de dados.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Arrays multidimensionais

- Tem duas ou mais dimensões
- Mantido como arrays de arrays de uma dimensão
  - Array com 2 dimensões: representa uma tabela
    - int[][] tabela = { {1, 2}, {3, 4};
      - tabela[linha][coluna];
        - tabela[0][0] = 1
        - tabela[0][1] = 2
        - tabela[1][0] = 3
        - tabela[1][1] = 4

    - int[][] b = new int[3][4]; // array 3 x 4

    - int[][] b = new int[2][]; // create 2 rows
    - b[0] = new int[5]; // create 5 columns for row 0
    - b[1] = new int[3]; // create 3 columns for row 1

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Métodos com parâmetros variáveis

- *variable-length argument lists:* ***elipsis(...)***
  - o método recebe um número variável de argumentos daquele tipo específico
  - pode ocorrer apenas uma vez em uma lista de parâmetros
  - as reticências, juntamente com seu tipo e o nome do parâmetro, devem ser colocadas no final da lista de parâmetros
  - alternativa mais consiva ao overloading e passar um array como parâmetro

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Parâmetros na linha de comando

- Já se perguntou o significa o String[] args como parâmetro do método main

  - **java InitArray 5 0 4**
    - args[0] = 5
    - args[1] = 0
    - args[4] = 4

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Classe java.util.Arrays

- java.util.Arrays: oferece um conjunto de métodos estáticos para manipulação de arrays
  - sort
  - binarySearch
  - equals
  - fill
  - ...

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Java API´s Collections classes

- ***java.util.ArrayList*** ***collection:*** ArrayList<E> → ***generic classes***
  - Permite o redimensionamento dinâmico, em tempo de execução
  - ArrayList<String> list;

Method	Description
`add`	Overloaded to add an element to the *end* of the `ArrayList` or at a specific index in the `ArrayList`.
`clear`	Removes all the elements from the `ArrayList`.
`contains`	Returns `true` if the `ArrayList` contains the specified element; otherwise, returns `false`.
`get`	Returns the element at the specified index.
`indexOf`	Returns the index of the first occurrence of the specified element in the `ArrayList`.
`remove`	Overloaded. Removes the first occurrence of the specified value or the element at the specified index.
`size`	Returns the number of elements stored in the `ArrayList`.
`trimToSize`	Trims the capacity of the `ArrayList` to the current number of elements.

Fig. 7.23 | Some methods of class `ArrayList<E>`.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Java API´s Collections classes

- ***collections***:
  - estrutura de dados predefinidas usadas para armazenar em memória grupos de objetos relacionados
  - Oferece métodos eficientes para organizar, armazenar e recuperar dados

- ***java.util.ArrayList*** ***collection:*** ArrayList<E> → ***generic classes***
  - Permite o redimensionamento dinâmico, em tempo de execução
  - ArrayList<String> list;

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Java API´s Collections classes

- ***generic classes***
  - Somente *referece types* podem ser usados para declarar variáveis e criar objetos de classes genéricas (*generic classes*)

    - ArrayList<Integer> integers;

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

3ª Leitura

INSTITUTO BRASILEIRO DE ENSINO, DESENVOLVIMENTO E PESQUISA

fabricio.santana@idp.edu.br
www.linkedin.com/in/fabriciofsantana